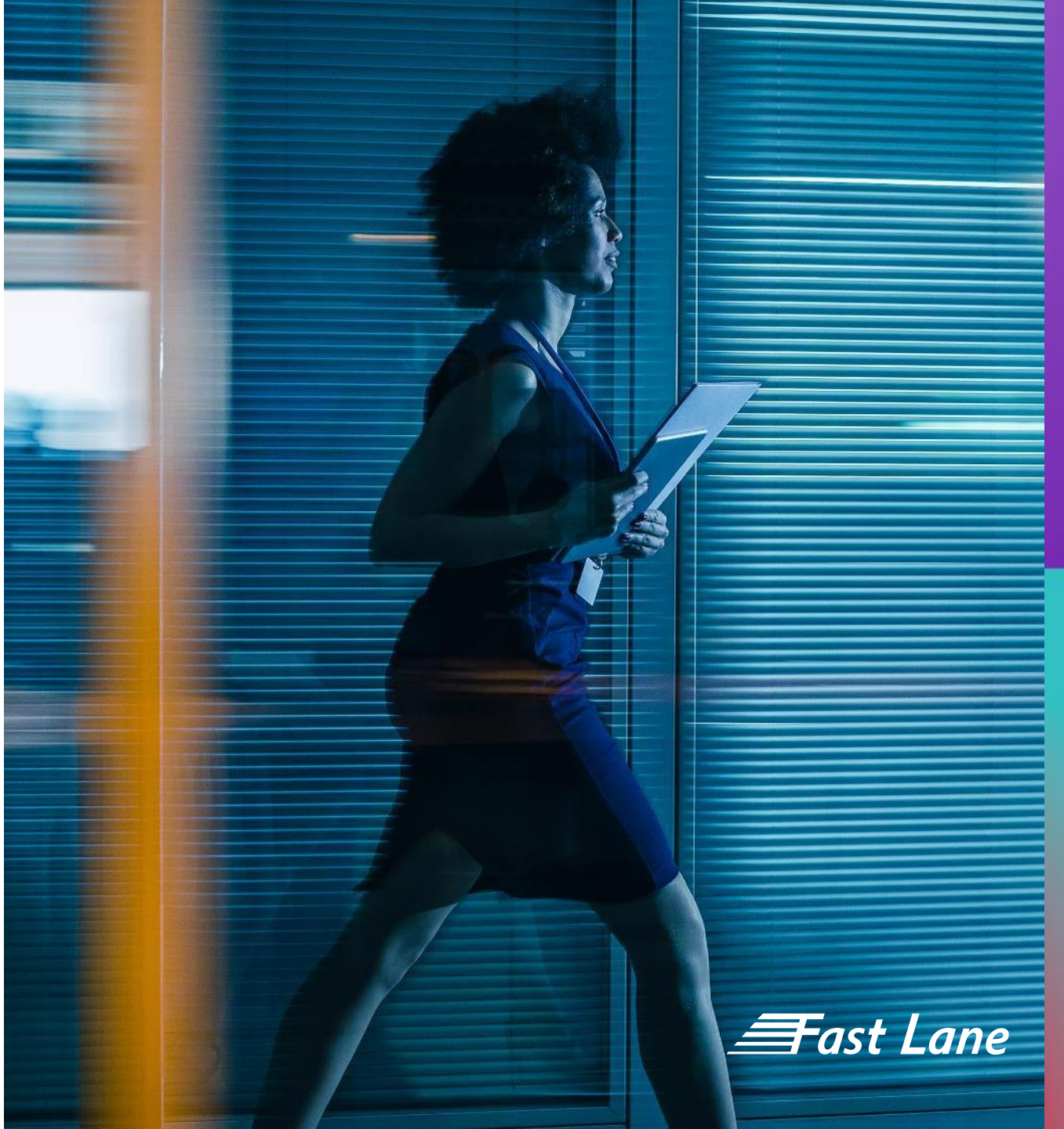
Fast Lane Group

Worldwide Education & Professional Services

Microsoft Solutions Partner

Microsoft Cloud Training Services



www.flane.de | © 2026 Fast Lane Institute for Knowledge Transfer GmbH

— AGENTIC AI: FROM STRATEGY TO **PRODUCTION.**

Alexander Kastil

Full stack, from enterprise software to agentic AI. Since 1996

AI at Work
Value discovery, KPI mapping and roadmap

Enterprise AI
Governed multi-agent systems on Microsoft Copilot and Foundry

Agentic Software Engineering
Harness architecture for Claude Code and GitHub Copilot

Team Enablement
AI driven content production and delivery

integrations.at
alexander.kastil@integrations.at

github.com/alexander-kastil
linkedin.com/in/alexander-kastil

INTEGRATIONS

AGENTIC AI CONSULTING · VIENNA



Microsoft CERTIFIED — AGENTIC AI BUSINESS SOLUTIONS ARCHITECT — ★★★

Microsoft CERTIFIED — AZURE AI APP AND AGENT DEVELOPER — ASSOCIATE ★★

Microsoft CERTIFIED — DEVOPS ENGINEER — EXPERT ★★★

SCAN TO CONNECT
integrations.at

MODULE 1

Designing Event-Driven Distributed Systems

Where the seams of Contoso Logistics belong before anyone writes a service

Event-Driven Distributed Apps on Azure: Aspire, Dapr & Durable Execution



01

Modular Monolith vs. Microservices

When a service boundary earns its cost

02

Sync vs. Async: Commands & Events

Request/reply, messages and temporal coupling

03

Event Storming: Find Service Boundaries

Bounded contexts and the Contoso service list

04

Event Types: Domain, Integration, CloudEvents

Payload styles and the CloudEvents 1.0 envelope

05

Reliability Patterns: Outbox, Saga, Idempotency

The failures that break event flows

06

Azure Messaging: Pick the Broker

Service Bus, Event Grid, Event Hubs, Dapr pub/sub

Designing event-driven distributed systems

Where the seams of Contoso Logistics belong before anyone writes a service

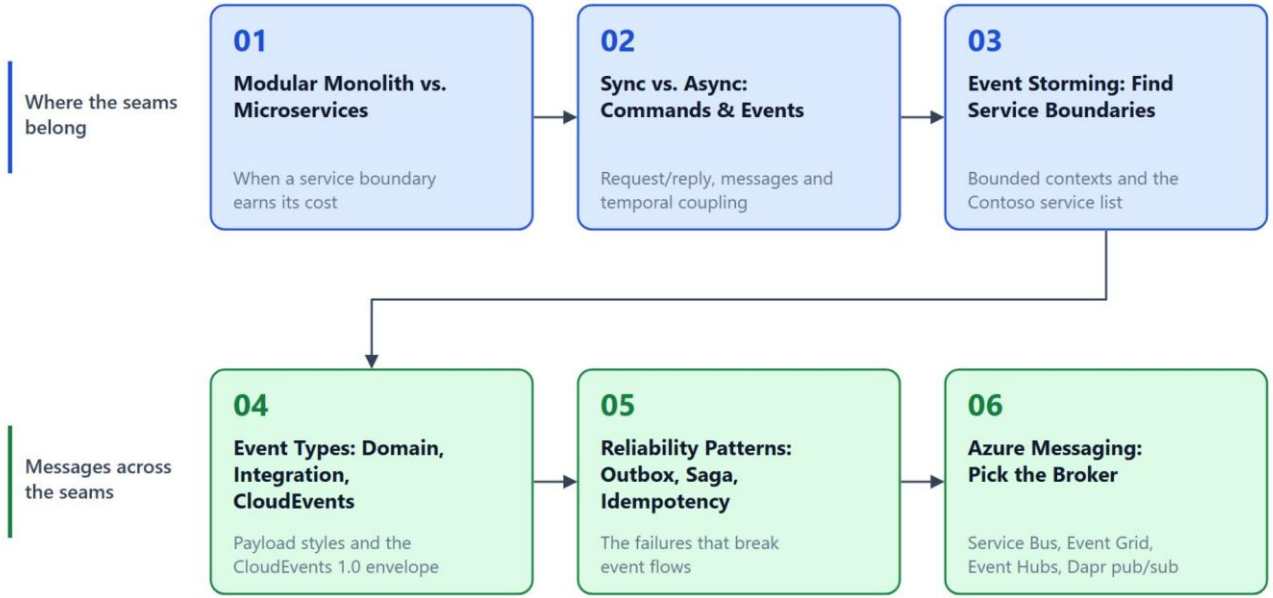
- **Boundaries**: a modular monolith or microservices, and event storming to find the seams
- **Messages**: HTTP or asynchronous commands and events, named and wrapped in CloudEvents
- **Reliability and brokers**: outbox, saga and idempotency, then one Azure broker per flow



Overview

The topics follow the order of a design decision

- Decide when a **modular monolith** is enough and when a split earns its cost
- Choose **HTTP** when the caller needs the answer, messages when the work only has to happen
- Find the boundaries with **event storming**, then name the events that cross them
- Keep each flow correct under failure with **reliability patterns**
- Match each flow to an **Azure broker**, then design the parcel flow in the lab



01

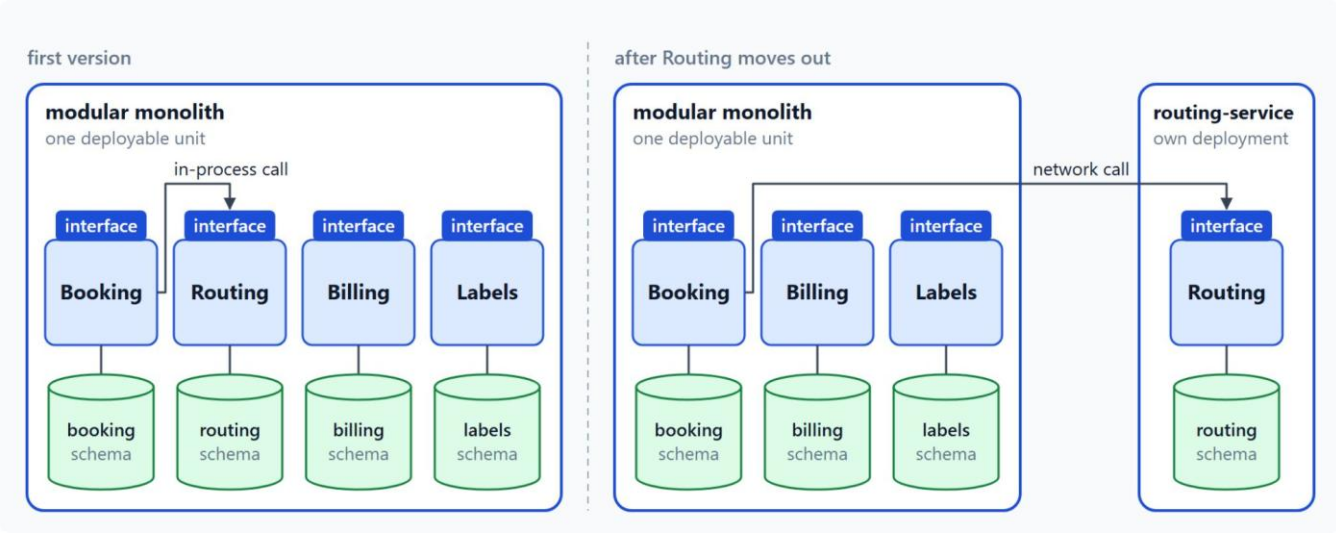
Modular Monolith vs. Microservices

When a service boundary earns its cost

A modular monolith as the starting point

Service boundaries inside one deployable unit

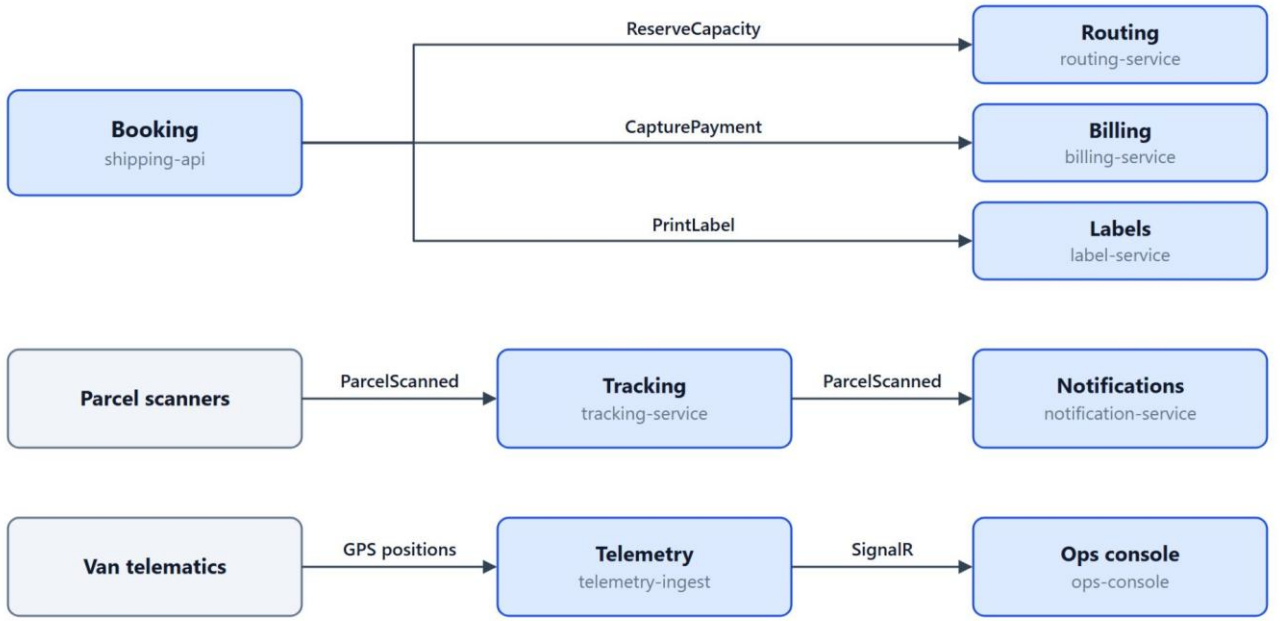
- Split for **team size**, independent deployment, scaling or data ownership
- A **modular monolith** deploys as one unit, organized into modules
- Each module owns its tables and exposes a small public interface
- Moving a module out turns its interface into the **service contract**



The cost of distribution

Each arrow between Contoso services is a call that can fail

- With **eventual consistency**, a booking can be confirmed before its van is assigned
- Booking calls routing and billing, and each **network call** can fail on its own
- Every service needs its own pipeline, dashboards and rollback path
- A new field on **ShipmentBooked** must not break consumers of the old shape
- Draw each boundary where the business language changes



02

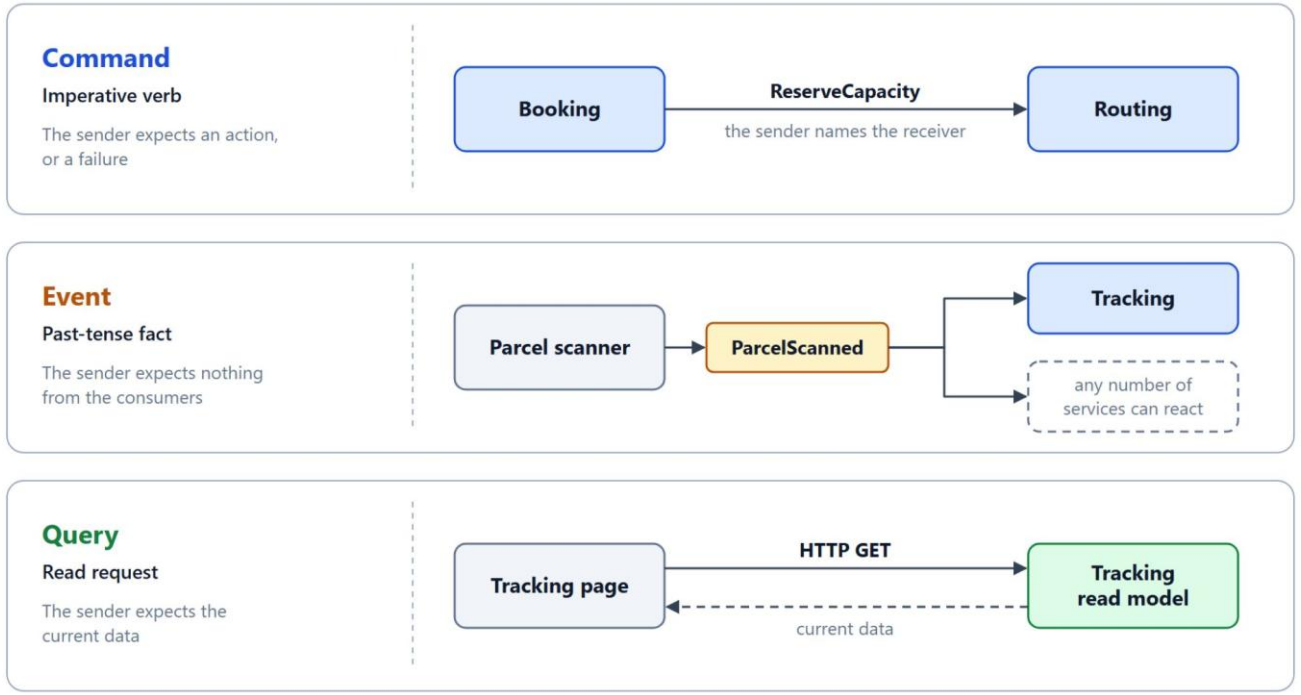
Sync vs. Async: Commands & Events

Request/reply, messages and temporal coupling

Commands, events and queries

What the sender expects decides the kind of message

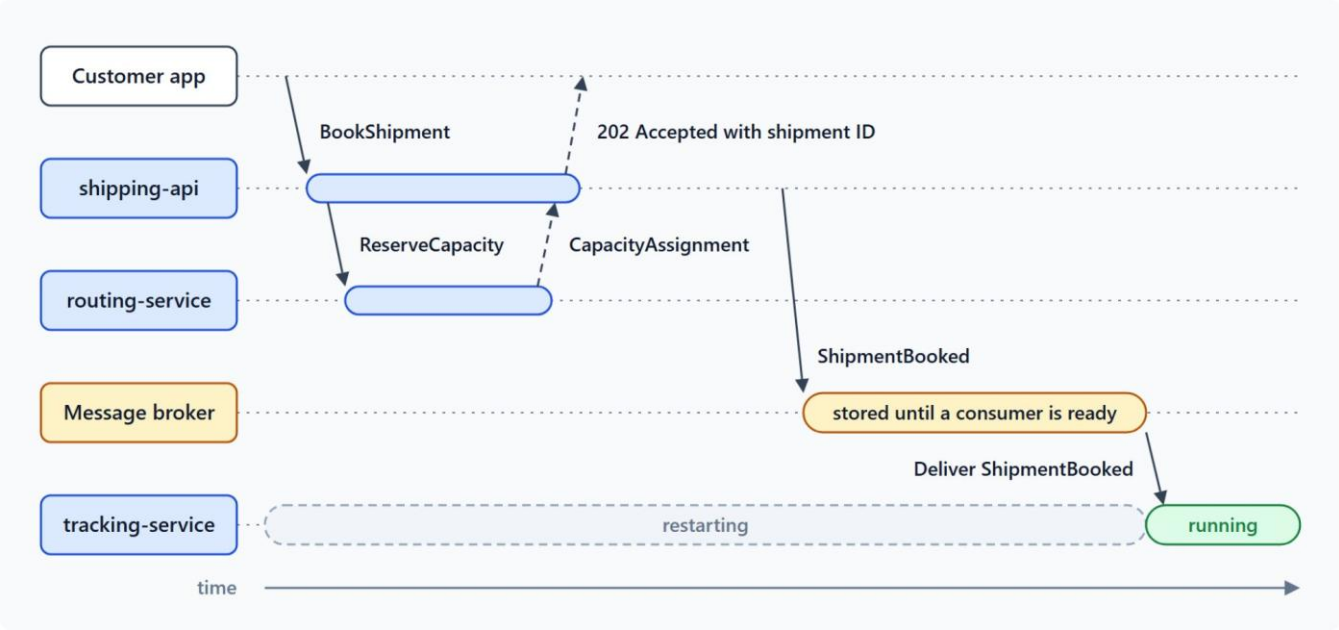
- A **command** asks a named service to act and expects an action or a failure
- An **event** reports a fact, and the publisher does not know who listens
- A **query** reads current data and changes nothing, so HTTP GET fits
- Commands use imperative verbs, events use the past tense



Request/reply and fire-and-forget

Both styles appear in one booking

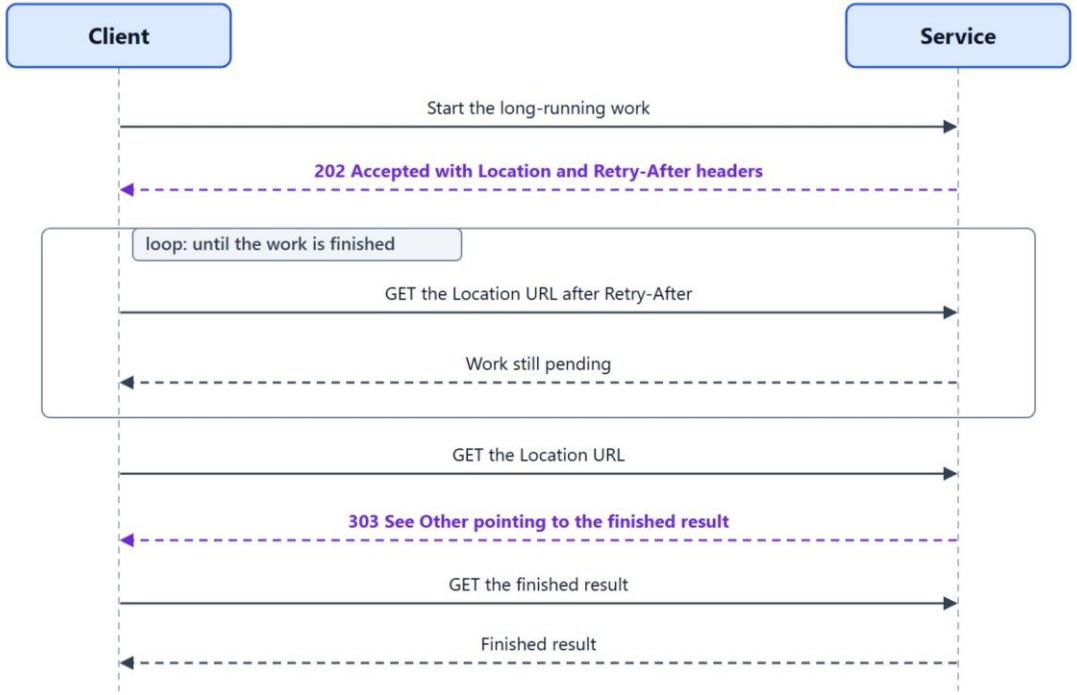
- With **request/reply** the caller waits, and HTTP is the default transport
- With **fire-and-forget** the broker stores the message until a consumer is ready
- Booking waits for ReserveCapacity, while ShipmentBooked waits in the broker
- **Temporal coupling** means both services must be up at the same moment
- Set each request/reply **timeout** from how long the customer will wait



When HTTP is the right choice

The Asynchronous Request-Reply pattern for long-running work

- The caller needs the answer for its next step, such as a **shipment ID**
- A **query** asks for current data
- Long-running work returns **202 Accepted** with Location and Retry-After
- The client polls until **303 See Other** points to the finished result
- If the caller only needs the work to happen, publish a command or an event



03

Event Storming: Find Service Boundaries

Bounded contexts and the Contoso service list

The sticky-note grammar

Business events on a wall, with the domain experts in the room

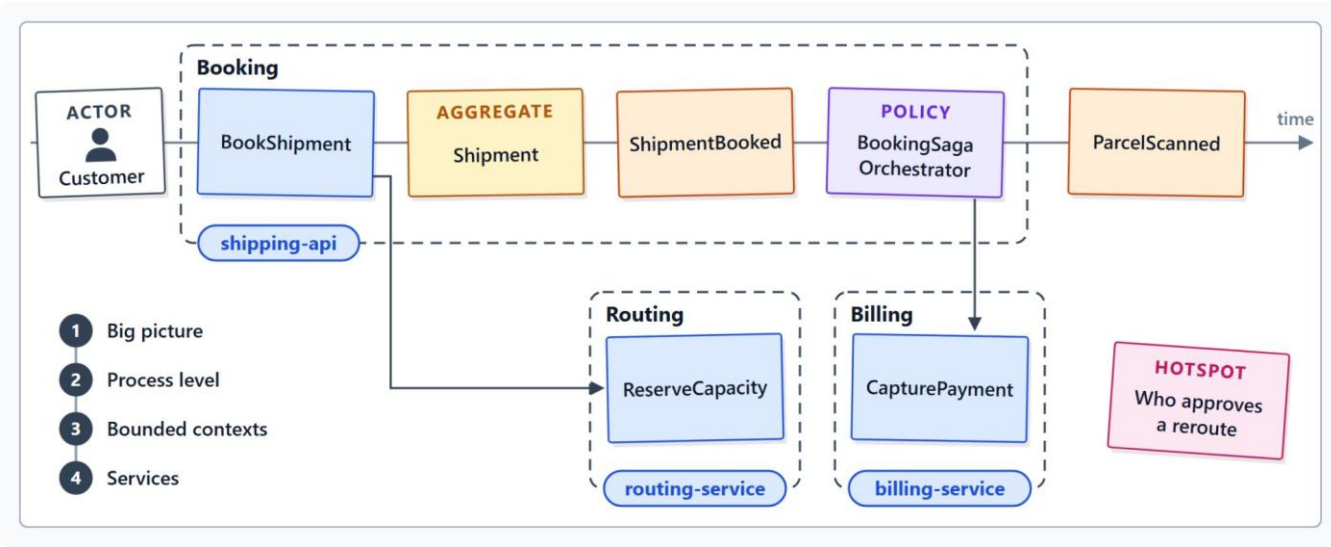
- **Events** are orange notes in the past tense, such as ShipmentBooked
- **Commands** are blue notes written as imperatives, such as BookShipment
- Actors issue commands, policies react to events, and **aggregates** decide
- **Hotspots** mark open questions, each with an owner and a date
- The big picture comes first, then the process level for one flow



From contexts to the Contoso service list

A microservice spans at most one bounded context

- In the **booking flow**, BookShipment invokes ReserveCapacity and the saga issues CapturePayment
- Aggregates that share one language form a **bounded context**
- A **context map** names the relationship between two contexts
- Eight Contoso contexts, each with its own service, such as Booking on shipping-api
- Each candidate passes the boundary tests or goes back to the storm



04

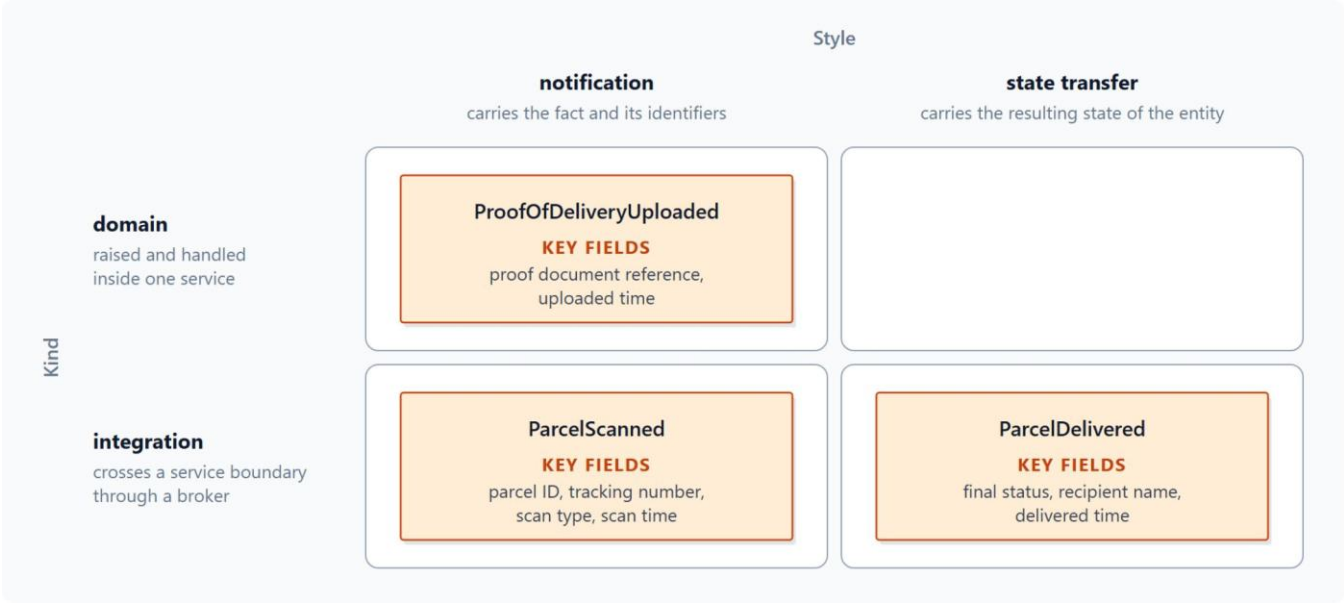
Event Types: Domain, Integration, CloudEvents

Payload styles and the CloudEvents 1.0 envelope

Domain events and integration events

The same fact inside one service or across a broker

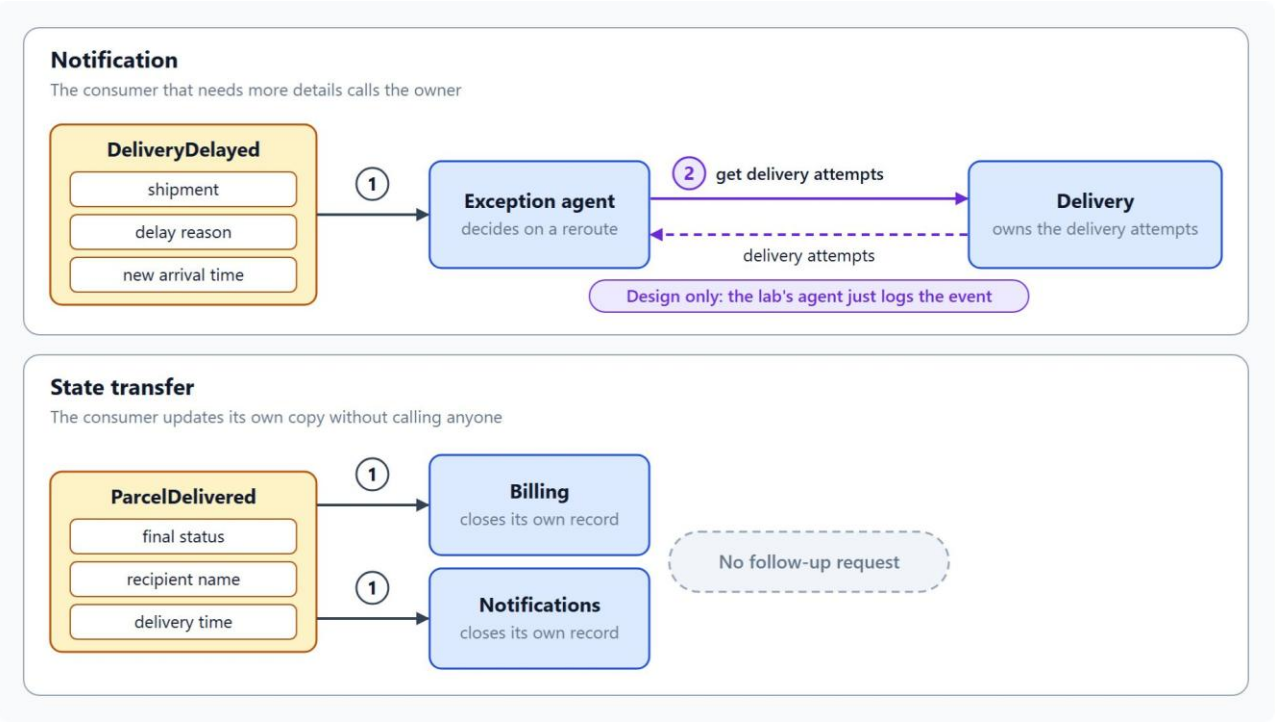
- A **domain event** is raised and handled inside one bounded context
- An **integration event** crosses a service boundary through a broker
- Integration events leave only after the entity is persisted
- The publisher owns the contract; the labs keep it in `Contoso.Contracts`
- Every event also has a style: notification or state transfer



Notification and state transfer events

What the payload holds decides whether the consumer asks back

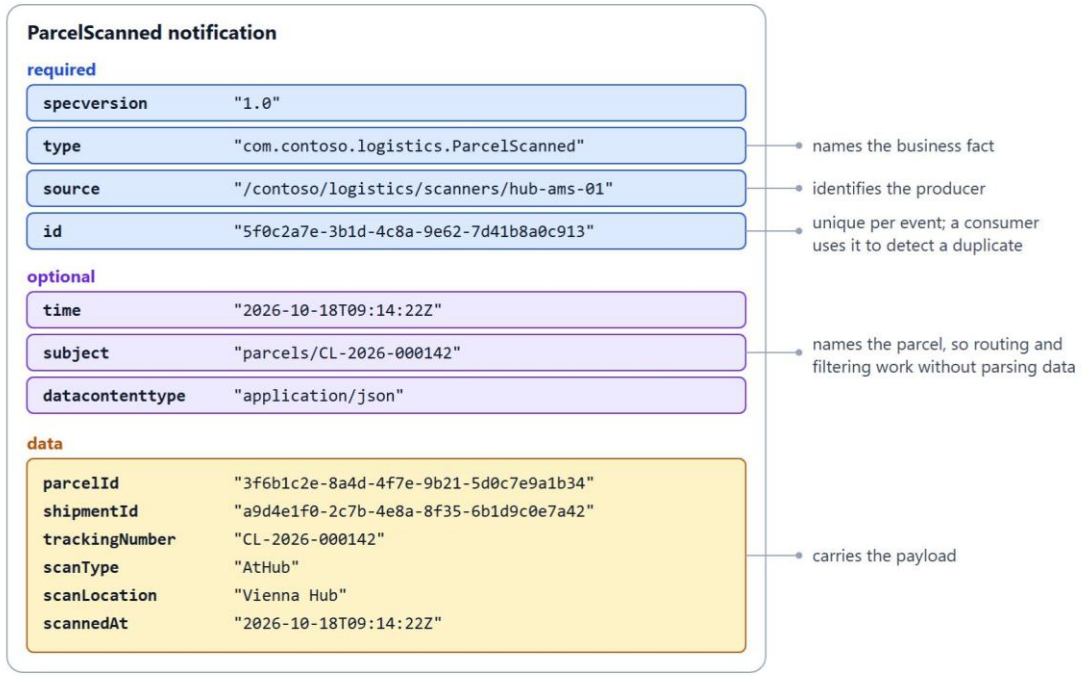
- A **notification** carries the fact, the identifiers and the time
- A consumer that needs more detail calls the owner in a second request
- A **state transfer** event carries the new state, and the consumer upserts its copy
- DeliveryDelayed is a notification, ParcelDelivered a state transfer
- Name events after business facts, such as **ParcelDelivered**



The CloudEvents 1.0 envelope

One envelope that consumers read without the producer's format

- Four required attributes: **id**, source, specversion and type
- Optional attributes are datacontenttype, subject and time; **data** carries the payload
- The id is unique per event, so consumers use it to detect duplicates
- The **subject** names the parcel, so brokers filter without parsing data
- Event Grid namespace topics and Dapr carry it; Service Bus carries plain JSON



05

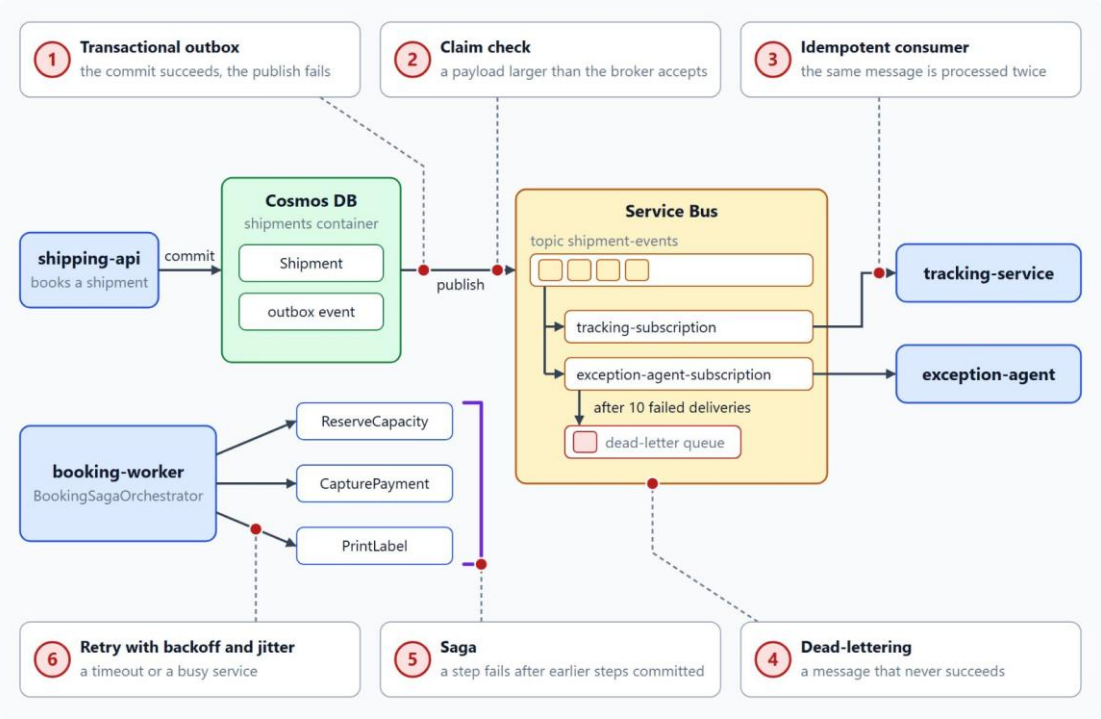
Reliability Patterns: Outbox, Saga, Idempotency

The failures that break event flows

Failures that break event flows

Each failure in one booking and the pattern that prevents it

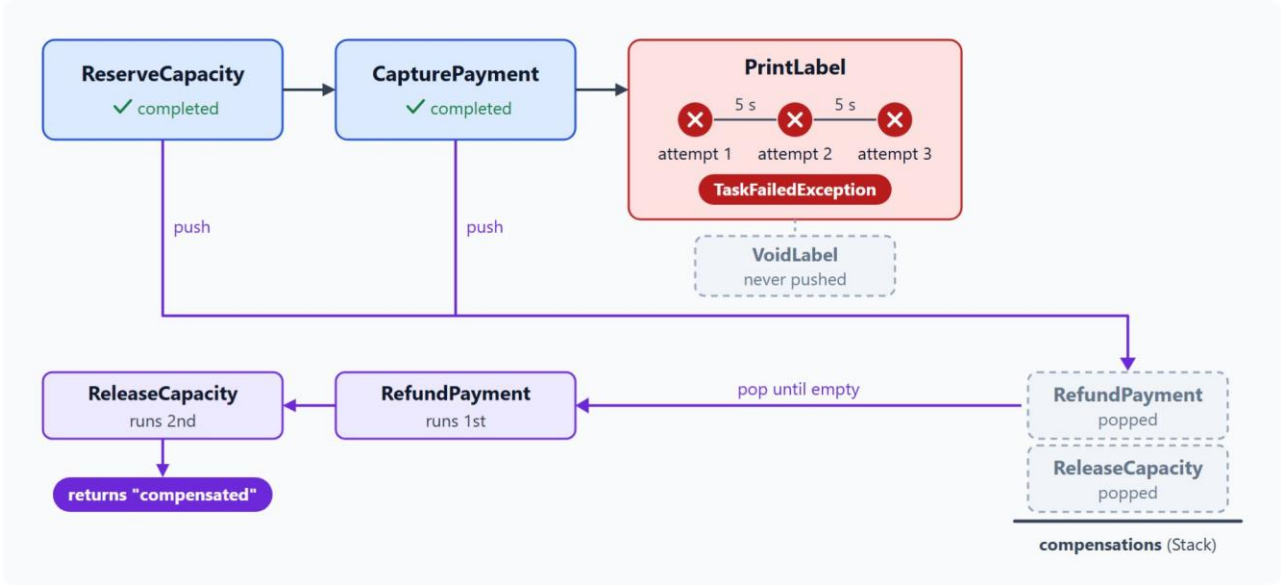
- The **transactional outbox** stores the change and the event in one transaction
- The **idempotent consumer** records each message it has processed
- A **saga** undoes committed steps when a later step fails
- **Retries** with exponential backoff and jitter handle transient faults
- **Dead-lettering** moves poison messages aside; a claim check carries large payloads



The booking saga

Local transactions with compensations in reverse order

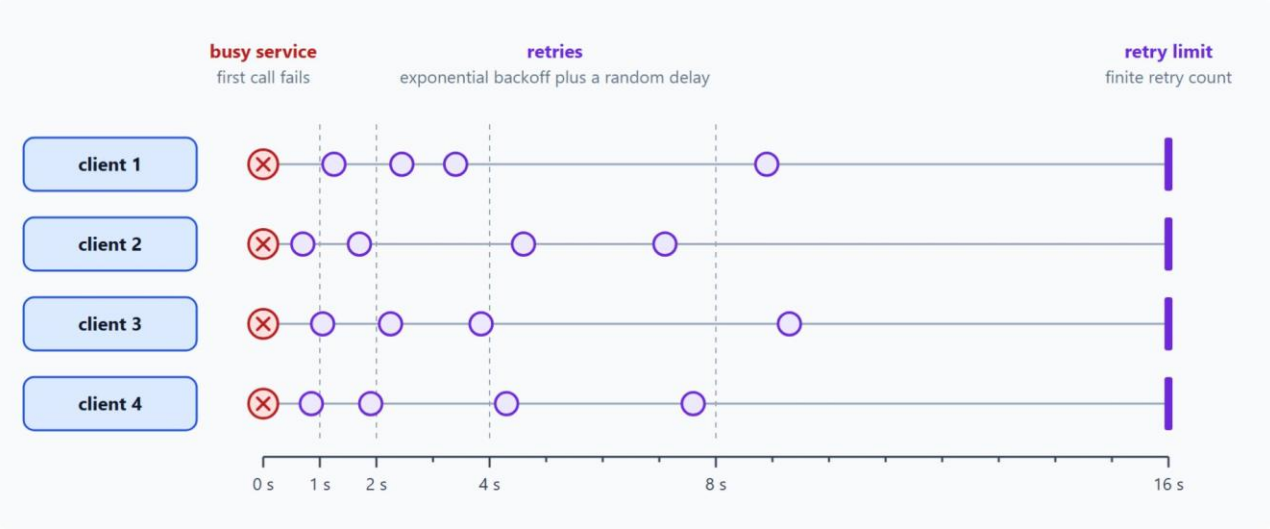
- A **saga** replaces one distributed transaction with local transactions
- Forward steps: ReserveCapacity, CapturePayment, PrintLabel
- **Compensations** release the capacity, refund the payment and void the label
- Compensations run in the reverse order of the forward steps
- Choreography fits a short flow, **orchestration** a growing one



Retries and dead-letters

Bounded retries for transient faults, a side queue for the rest

- Retry transient faults with **exponential backoff and jitter**
- Keep a finite retry count and a retry budget per dependency
- Give every outbound call a **timeout**
- Service Bus dead-letters a message after **ten** failed deliveries by default
- Event Grid dead-letters to a storage account once you configure it



06

Azure Messaging: Pick the Broker

Service Bus, Event Grid, Event Hubs, Dapr pub/sub

Messages, events and streams

Service Bus, Event Grid, Event Hubs and Dapr pub/sub compared

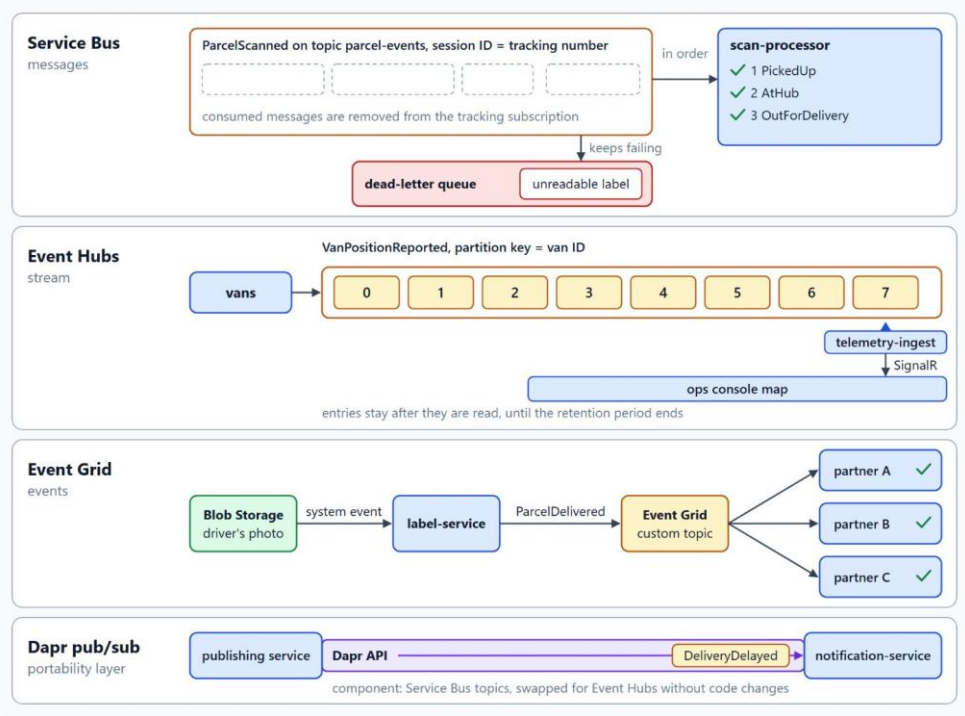
- A **message** carries data under a contract, the shape Service Bus fits
- An **event** is a lightweight notification of a state change, for Event Grid
- A **stream** is a time-ordered series each consumer reads at its own pace, for Event Hubs
- **Dapr pub/sub** is a portability layer that sits on top of brokers
- Ordering, retention and dead-lettering decide most designs

	Service Bus messages	**Event Grid** events	**Event Hubs** stream	**Dapr pub/sub** portability layer
Ordering	FIFO within a session	No guarantee	Per partition	Component-dependent
Retention	Until the message time to live expires	Custom topic TTL of 1 to 1440 minutes; namespace topic up to 7 days	1 to 90 days, depending on tier	Component-dependent
Dead-letter	Dead-letter sub-queue	Storage account, when configured	None built in	Component-dependent

Contoso traffic on each service

Three Azure brokers side by side, Dapr pub/sub on the notification path

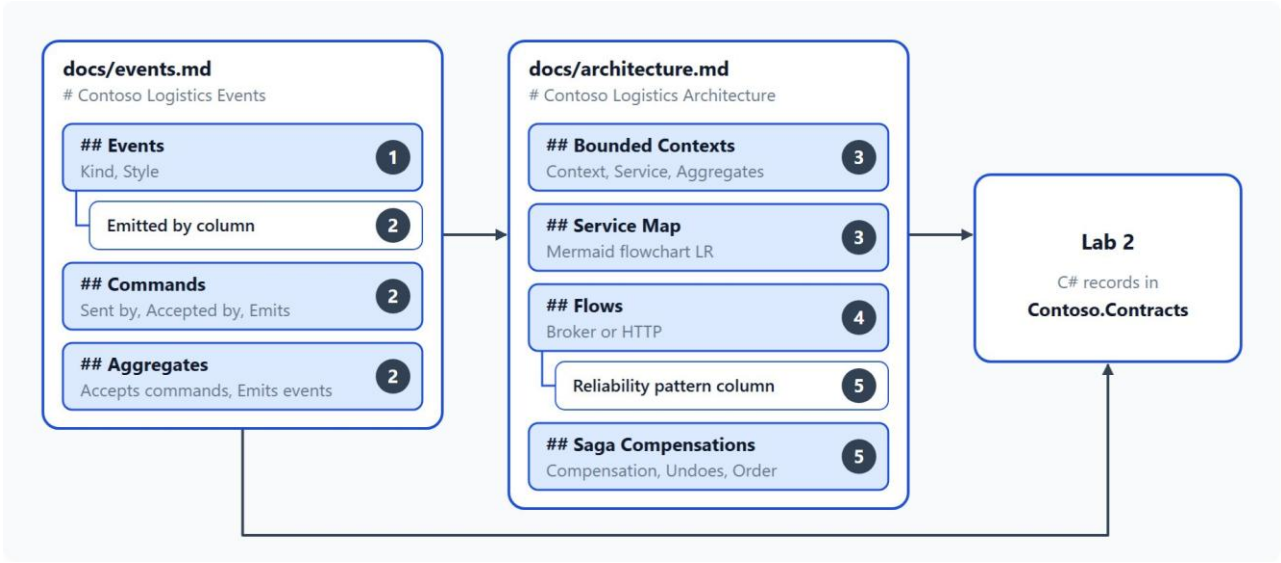
- **Service Bus** carries the booking commands and ParcelScanned, one session per tracking number
- **Event Hubs** carries VanPositionReported over Kafka, keyed by van ID
- **Event Grid** sends ParcelDelivered to partners after the driver's photo upload
- **Dapr pub/sub** carries DeliveryDelayed to notification-service
- Check ordering first, then retention



Lab: event-storm the parcel delivery flow

Two design documents that lab 2 turns into C# records

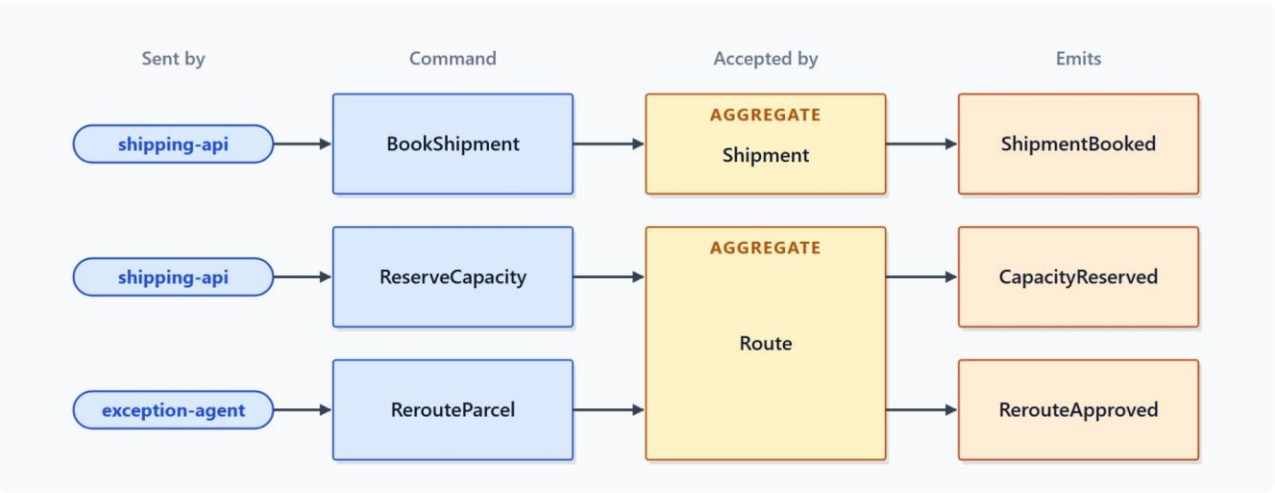
- Starts from an empty workspace with agent instructions and an empty `docs/` folder
- `docs/events.md` holds 13 events, 7 commands and the aggregates behind them
- `docs/architecture.md` holds contexts, the service map, flows and saga compensations
- Five steps in about 45 minutes, with no code and no tests
- **Design checks** confirm both documents before lab 2



Events, commands and aggregates in the lab

Steps 1 and 2 write docs/events.md

- Step 1 lists 13 events, each with a **kind** and a **style**
- ParcelScanned carries the scan types PickedUp, AtHub and OutForDelivery
- Step 2 adds an Emitted by column and 7 **commands**
- Every Accepted by value matches a row in the Aggregates table
- Steps 3 to 5 draw the contexts, pick a broker per flow and add the patterns



Takeaways

What to carry into the Aspire module

- Start with a modular monolith and split when team, deployment, scaling or data demand it
- Use HTTP when the caller needs the answer, messages when the work only has to happen
- Draw boundaries where the business language changes
- Publish integration events after the commit, named after business facts
- Protect every flow with a pattern, and pick the broker by ordering, then retention

